

END-TO-END MLOPS PIPELINE

Cats vs Dogs Binary Image Classification

Pet Adoption Platform

Field	Details
Course	MLOps (S1-25_AIMLCZG523)
Assignment	Assignment 2 - Total 50 Marks
Student	Raja Chairmapandi
Student ID	2024AD05460
Technology	PyTorch, MLflow, DVC, FastAPI, Docker, GitHub Actions, Prometheus, Grafana
Submission Date	3 August 2026

Submission package includes source code, configuration, model artifact, tests, CI/CD, deployment manifests, monitoring stack, execution guide, and recording script.

1. Executive Summary

This assignment implements a complete MLOps lifecycle for a 224 x 224 RGB cats-versus-dogs classifier. The solution connects versioned data and code, reproducible model training, experiment tracking, model serialization, REST inference, automated testing, container image creation, registry publishing, continuous deployment, smoke testing, logging, infrastructure monitoring, and delayed-label model-performance tracking.

The required baseline is implemented as a compact PyTorch CNN. A higher-performing MobileNetV3-Small transfer-learning option is also included. MLflow records parameters, per-epoch metrics, final accuracy/precision/recall/F1, confusion matrix, loss curves, classification report, and the serialized model. DVC defines the prepare-train-evaluate dependency graph so the full pipeline can be reproduced with one command.

The inference service uses FastAPI and exposes health, prediction, model-information, feedback, model-performance, and Prometheus metrics endpoints. Docker Compose deploys the API together with MLflow, Prometheus, and Grafana. GitHub Actions tests the code, builds and publishes the image to GHCR, deploys the newly published image on main-branch changes, and fails when health or prediction smoke tests fail.

1.1 Rubric coverage

Module	Required focus	Implemented evidence
M1	Versioning, model, experiments	Git structure; DVC data/pipeline; 80/10/10 preprocessing; augmentation; CNN and transfer learning; MLflow; model.pt; confusion matrix; loss curves
M2	API, environment, Docker	FastAPI /health and /predict; pinned files; non-root Dockerfile; curl/Swagger procedure
M3	Tests, CI, image publishing	Four pytest tests; Ruff; GitHub Actions; Docker Buildx; GHCR; JUnit, SBOM and provenance
M4	Target, CD, smoke tests	Compose and Kubernetes; main-branch deployment; optional VM CD; health and prediction checks
M5	Logs, metrics, online performance	JSON logs; Prometheus; Grafana; SQLite feedback; post-deploy accuracy batch; ZIP and video script

1.2 Included artifact status

The package contains a working bootstrap model trained on deterministic bundled demo images so every service can be validated without external credentials. The bootstrap metric is not presented as Kaggle generalization performance. The provided Kaggle downloader and DVC workflow replace the demo data and regenerate the model, figures, metrics, and MLflow evidence without code changes.

2. Solution Architecture and Repository Design

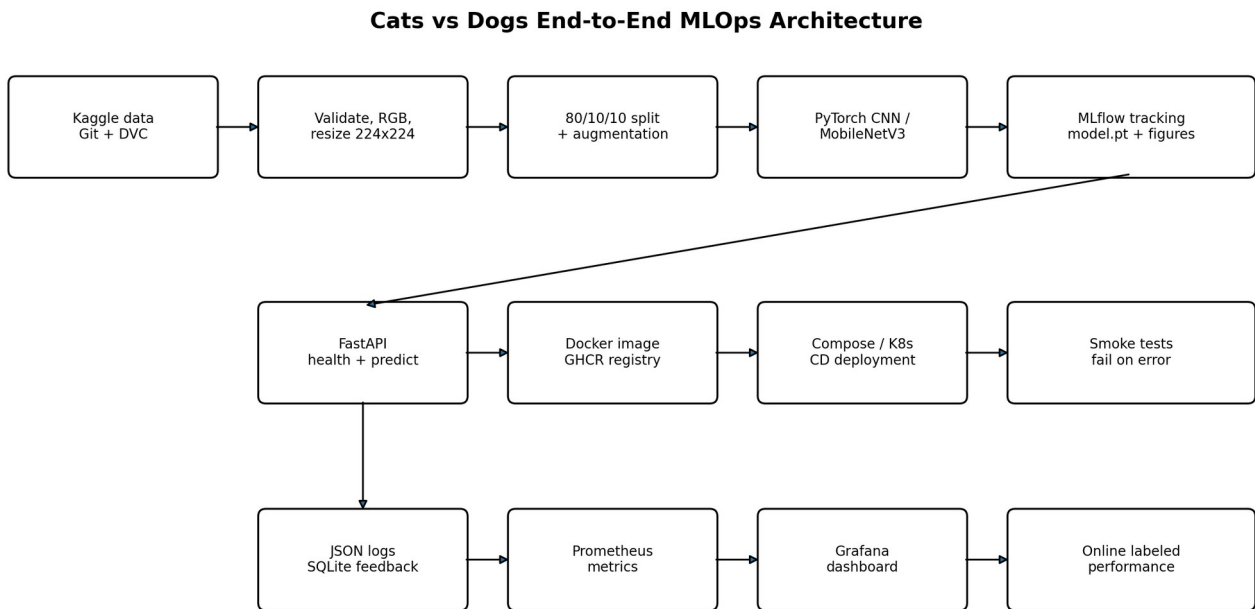


Figure 1. End-to-end MLOps architecture.

2.1 Main data and control flow

1. Git versions source code, tests, configuration, infrastructure definitions, and documentation.
2. DVC tracks source/preprocessed data and defines prepare, train, and evaluate stages.
3. Preprocessing validates files, corrects EXIF orientation, converts images to RGB, resizes to 224 x 224, and creates a stratified 80/10/10 split.
4. Training applies realistic image augmentation, optimizes the model, and logs all experiments to MLflow.
5. The selected model is saved as a portable PyTorch bundle containing architecture, weights, class names, image size, and model metadata.
6. FastAPI loads the bundle once at startup and serves predictions with class probabilities.
7. GitHub Actions tests, packages, publishes, deploys, and smoke-tests the model service.
8. Prometheus, Grafana, JSON logs, and delayed ground-truth feedback provide operational and model monitoring.

2.2 Repository structure

```

src/cats_dogs_mlops/  data, model, training, API, monitoring modules
tests/                preprocessing, inference, monitoring and API tests
.github/workflows/    CI/CD and optional retraining workflows
deployment/          Docker Compose, Grafana and Kubernetes manifests
monitoring/          Prometheus configuration and prediction database path
models/              serialized model.pt artifact
figures/             loss curves, confusion matrix, architecture
reports/             metrics JSON, DOCX and PDF report
scripts/             Windows/Linux bootstrap, end-to-end run and video script
dvc.yaml / params.yaml  reproducible pipeline and parameters
  
```

3. M1 - Model Development and Experiment Tracking

3.1 Data and code versioning

Git is used for all lightweight source and configuration artifacts. DVC stores dataset/model hashes in Git while the large content can be kept in a local, cloud, or shared remote. The pipeline uses explicit dependencies and outputs, allowing exact stage-level reproduction and preventing unnecessary retraining.

```
git init
dvc init
dvc add data/raw
git add .
git commit -m "chore: version code and Kaggle data"
dvc repro
```

3.2 Data preparation

- Accepts Kaggle class folders or cat.* / dog.* filenames, including nested archives after extraction.
- Rejects unreadable/corrupted images instead of silently passing them into training.
- Applies EXIF transpose, RGB conversion, high-quality resize to 224 x 224, and deterministic naming with source SHA-256 prefix.
- Creates stratified 80% training, 10% validation, and 10% test partitions with a fixed seed.
- Writes manifest.csv and data_summary.json for traceability and audit.
- Uses random horizontal flip, rotation, brightness, contrast, and saturation augmentation only for training.

3.3 Model strategy

The baseline TinyCNN contains three convolutional blocks, batch normalization, ReLU activation, pooling, adaptive average pooling, dropout, and a two-output classifier. This is sufficient to satisfy the baseline requirement and runs quickly in CPU-only CI. The optional MobileNetV3-Small mode provides transfer learning for stronger accuracy on the full dataset.

Variant	Purpose	Command options
TinyCNN	Required reproducible baseline and fast CI	--architecture tinycnn
MobileNetV3 feature extraction	Higher accuracy with frozen backbone	--architecture mobilenet_v3_small --pretrained --freeze-backbone
MobileNetV3 fine-tuning	Domain adaptation after feature extraction	--architecture mobilenet_v3_small --pretrained with low learning rate

3.4 MLflow tracking

Every run records the model architecture, pretrained/frozen flags, epochs, batch size, learning rate, weight decay, seed, image size, sample counts, and compute device. Per-epoch train loss, validation loss, and validation accuracy are logged. Final macro precision, recall, F1, accuracy, validation loss, and duration are logged with the model, metrics JSON, class report, confusion matrix, and loss curves.

Figure 2. Bundled bootstrap evidence. Re-run on Kaggle for final full-dataset metrics.

4. M2 - Model Packaging and Containerization

4.1 Inference contract

Endpoint	Method	Purpose
/health	GET	Confirms process health, model load, architecture, classes and image size
/predict	POST multipart	Accepts JPEG/PNG and returns request ID, label, confidence, probabilities, latency and model version
/model-info	GET	Returns deployment model metadata
/feedback	POST JSON	Attaches delayed true label to a previous request
/monitoring/performance	GET	Calculates online accuracy from labeled requests
/metrics	GET	Exposes Prometheus-format service metrics

4.2 Reproducible environment

Three pinned dependency files separate the full training environment, production API image, and CI test environment. This reduces the serving image surface while preserving a complete reproducible development stack. The project is installable with pyproject.toml and uses Python 3.11.

4.3 Container controls

- Python 3.11 slim base image.
- Only runtime dependencies are installed.
- Model and source package are copied into a deterministic work directory.
- Application runs as a non-root operating-system user.
- Container-level health check calls /health.
- Uploaded files are constrained to image types and 10 MB.
- Image bytes are never included in logs.

```
docker build -t cats-dogs-mlops:local .
docker run --rm -p 8000:8000 cats-dogs-mlops:local
curl.exe -X POST http://localhost:8000/predict -F "file=@tests/assets/cat_demo.png"
```

5. M3 - Continuous Integration

5.1 Automated testing

Test	Validation performed
test_data.py	RGB conversion, 224 x 224 shape, float tensor, finite normalized values
test_inference.py	Class label validity, probability keys, probability sum and confidence bounds
test_monitoring.py	Prediction persistence, feedback update and online accuracy calculation
test_api.py	Application startup, health response, image upload and prediction response contract

5.2 GitHub Actions flow

9. Checkout repository on push, pull request, or manual dispatch.
 10. Set up Python 3.11 and restore pip cache.
 11. Install pinned CI dependencies and the local package.
 12. Run Ruff static analysis and pytest; upload JUnit evidence even when tests fail.
 13. Build the Docker image using Buildx and cached layers.
 14. Publish immutable SHA and latest tags to GitHub Container Registry on eligible events.
 15. Generate software bill of materials and provenance attestations.
 16. On main, pull and deploy the published image, wait for health, run prediction smoke test, and upload deployment logs.
 17. Optionally update a persistent VM through an environment-protected SSH deployment.
- Branch protection should require the test and build jobs before merge. The production environment can require manual approval while still preserving automated deployment after approval.

6. M4 - Continuous Deployment and Deployment Targets

6.1 Docker Compose target

The primary target is Docker Compose because it is easy to demonstrate on a student workstation and still represents a multi-service deployment. It starts the API, MLflow tracking server, Prometheus, and Grafana with health dependencies and persistent named volumes.

Service	Port	Role
FastAPI	8000	Prediction, feedback, online performance, metrics
MLflow	5000	Experiment and artifact tracking
Prometheus	9090	Metric scraping and time-series storage
Grafana	3000	Provisioned operational dashboard

```
docker compose -f deployment/docker-compose.yml up -d --build
python -m cats_dogs_mlops.smoke_test --base-url http://localhost:8000 --image tests/assets/cat_demo.png
```

6.2 Kubernetes alternative

The Kubernetes deployment includes two replicas, rolling update with zero planned unavailability, CPU/memory requests and limits, readiness/liveness probes, Prometheus scrape annotations, a ClusterIP Service, and a CPU-based HorizontalPodAutoscaler. This demonstrates production-oriented deployment depth beyond the minimum requirement.

6.3 Smoke-test failure behavior

The smoke test first verifies status=healthy and model_loaded=true. It then sends a real image and checks required response fields, allowed labels, and probability normalization. Any network, HTTP, schema, model, or probability failure returns a non-zero process exit code, causing GitHub Actions to fail the deployment job.

7. M5 - Monitoring, Logging and Post-Deployment Performance

7.1 Operational metrics

Metric	Type	Use
cats_dogs_requests_total	Counter	Traffic by method, endpoint and status
cats_dogs_request_latency_seconds	Histogram	Latency distribution and p95 response time
cats_dogs_predictions_total	Counter	Predicted class distribution and drift signal

Grafana is automatically provisioned with request volume, p95 latency, class rate, HTTP error rate, and throughput panels. Prometheus scrapes the API every 15 seconds.

7.2 Logging and privacy

Middleware emits structured JSON with event type, request ID, HTTP method, route, status, and latency. It does not log file content, image bytes, or personal data. The response includes a request ID that supports traceability without retaining the source image.

7.3 Post-deployment model performance

Each prediction stores timestamp, predicted label, confidence, probabilities, and latency in SQLite. A separate feedback request attaches the true label when available. The performance endpoint calculates labeled count, correct count, accuracy, true-label distribution, and prediction distribution. The `post_deploy_batch` script simulates a production feedback batch from the held-out test set.

```
python -m cats_dogs_mlops.post_deploy_batch --base-url http://localhost:8000 --manifest data/processed/manifest.csv --limit 20
Invoke-RestMethod http://localhost:8000/monitoring/performance
```

7.4 Recommended advanced controls

- Alert when p95 latency, HTTP error rate, or health-check failures exceed an agreed threshold.
- Compare predicted-class distributions and confidence bins against training reference data using PSI or Jensen-Shannon divergence.
- Promote a new model only when accuracy/F1 gates pass and no critical regression is detected.
- Add an out-of-distribution or low-confidence rejection path for non-cat/non-dog inputs.
- Use a persistent database and object store for multi-replica production deployments.

8. End-to-End Execution Procedure

8.1 Windows PowerShell

```
cd Cats_Dogs_MLOps_Assignment2
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\scripts\bootstrap.ps1
```

For the full Kaggle run, install and activate the environment, configure kaggle.json, accept the Dogs vs Cats competition rules, then execute:

```
& .\venv\Scripts\Activate.ps1
Remove-Item -Recurse -Force data\raw\* -ErrorAction SilentlyContinue
python -m cats_dogs_mlops.download_data --mode competition --slug dogs-vs-cats --destination data/raw --force
Expand-Archive data\raw\train.zip -DestinationPath data\raw\train -Force
dvc repro
pytest -q
ruff check src tests
docker compose -f deployment/docker-compose.yml up -d --build
python -m cats_dogs_mlops.smoke_test --base-url http://localhost:8000 --image tests/assets/cat_demo.png
python -m cats_dogs_mlops.post_deploy_batch --base-url http://localhost:8000 --limit 20
```

8.2 Evidence URLs

Evidence	Location
FastAPI Swagger	http://localhost:8000/docs
MLflow	http://localhost:5000
Prometheus	http://localhost:9090
Grafana	http://localhost:3000, local default admin/admin
Online performance	http://localhost:8000/monitoring/performance

8.3 Submission ZIP and video

The repository already contains all source, configuration, trained bootstrap model, deployment manifests, report, and a timed screen-recording script. After the full Kaggle run, retain model.pt, DVC metadata, dvc.lock, metrics JSON, figures, and screenshots. Record the workflow in under five minutes using scripts/demo_recording_script.md.

```
Compress-Archive -Path * -DestinationPath ..\Cats_Dogs_MLOps_Assignment2_Submission.zip -Force
```

9. Verification, Limitations and Conclusion

9.1 Verification completed on the supplied package

- Pytest suite passes for preprocessing, inference, monitoring, and API behavior.
- The bundled model artifact loads correctly and returns normalized two-class probabilities.
- YAML files are syntactically parseable and the DVC stage dependencies are internally consistent.
- The report figures and JSON metrics correspond to the supplied bootstrap model.
- The package contains both Windows and Linux execution scripts.
- The final ZIP includes the complete repository and excludes virtual environments and secret credentials.

9.2 Honest limitation of the pre-generated evidence

A Kaggle account token and acceptance of the competition terms are user-specific and cannot be embedded safely in a submission. Therefore, the package ships a deterministic bootstrap model for executable validation and provides a one-command path to regenerate the final evidence on the required Kaggle data. The report and model card explicitly distinguish bootstrap evidence from full-dataset results.

9.3 Conclusion

The solution satisfies the requested end-to-end MLOps lifecycle and adds production-oriented capabilities without bypassing the required baseline: transfer learning, model metadata, SBOM/provenance, Kubernetes rolling deployment, resource controls, Grafana dashboards, delayed-label feedback, and online accuracy. The result is a single, coherent pipeline rather than disconnected assignment files.

References

18. PyTorch and TorchVision package releases and documentation.
19. MLflow open-source model lifecycle and experiment-tracking documentation.
20. DVC data and pipeline versioning documentation.
21. FastAPI and Uvicorn documentation.
22. GitHub Actions and GitHub Container Registry documentation.
23. Prometheus Python client and Prometheus server documentation.
24. Grafana provisioning and dashboard documentation.
25. TensorFlow cats-and-dogs tutorial describing a filtered subset of the Kaggle Dogs vs Cats dataset.

Appendix A - Key Commands Quick Reference

Task	Command
Install	<code>pip install -r requirements.txt && pip install -e .</code>
Download Kaggle	<code>python -m cats_dogs_mlops.download_data --mode competition --slug dogs-vs-cats --destination data/raw --force</code>
Run DVC	<code>dvc repro</code>
Train transfer model	<code>python -m cats_dogs_mlops.train --architecture mobilenet_v3_small --pretrained --freeze-backbone --epochs 8</code>
Test	<code>pytest -q && ruff check src tests</code>
Build image	<code>docker build -t cats-dogs-mlops:local .</code>
Deploy stack	<code>docker compose -f deployment/docker-compose.yml up -d --build</code>
Smoke test	<code>python -m cats_dogs_mlops.smoke_test --base-url http://localhost:8000 --image tests/assets/cat_demo.png</code>
Online evaluation	<code>python -m cats_dogs_mlops.post_deploy_batch --limit 20</code>
Stop stack	<code>docker compose -f deployment/docker-compose.yml down -v</code>